
HOW_TO.pdf - Scenario A

Autonomous Egg Presence Monitoring

---

1. Prerequisites

Hardware

- Raspberry Pi 3
- MicroSD (≥32 GB)
- ESP32-CAM
- 2 × LilyGO TTGO LoRa 868 MHz
- USB cables
- Power Bank (5V / 2.4A)
- Wi-Fi connection
- PC with Arduino IDE
- PC running Node-RED

Software

- Raspberry Pi Imager
- Arduino IDE
- Python 3
- Node-RED

---

2. Raspberry Pi Configuration

Install Raspberry Pi OS

1. Install Raspberry Pi Imager https://www.raspberrypi.com/software/
2. Insert the microSD.
3. Select:

   ➔ Device:

   Raspberry Pi 3

➔ OS:

Raspberry Pi OS (64-bit)

➔ Storage:

MicroSD

Before writing the image:

Enable

- SSH
- Wi-Fi
- Username
- Password

Connect via SSH

1. Find the Raspberry IP by wiriting on your terminal hostname -I

   Example: 192.168.0.108

2. Connect: ssh davide4ne@192.168.0.108 (put your username instead of davide4ne)

---

3. Create Python Virtual Environment

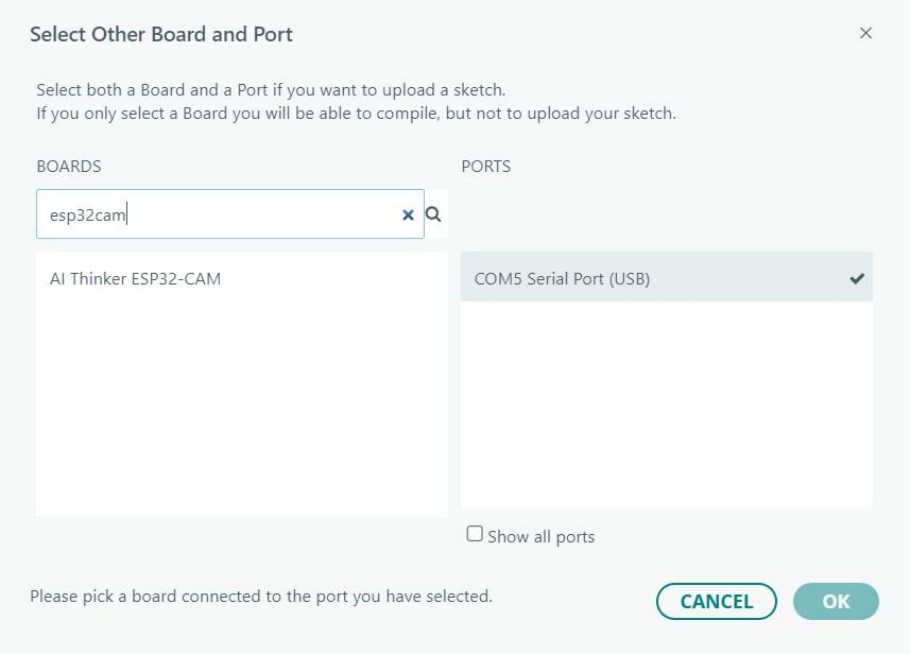
1. python3 -m venv venv
2. Activate: source venv/bin/activate
3. Install dependencies: pip install -r requirements.txt

---

4. ESP32-CAM Configuration

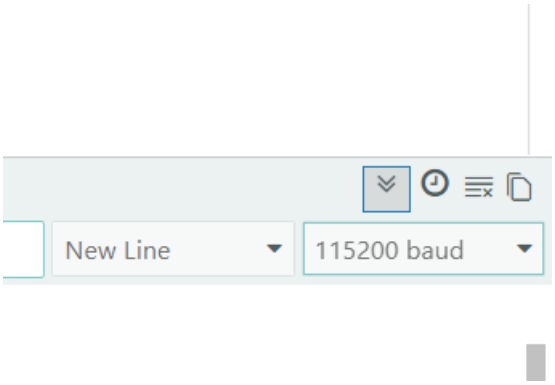
First of all, connect your ESP32-CAM to your laptop by using the micro-usb cable. Then:

1. Open Arduino IDE.
2. Install:
   - ESP32 board package
   - Required libraries you can find on README.md
3. Open ESP32_CAM.ino
4. Select the right COM. You can find it on tools -> port -> COM*
5. On the top, select the right board (you can look at the COM*)



6. Compile.
7. Upload.

You can look at the results of your upload by clicking on “Serial Monitor” at the top right of the interface. Pay attention about the baud rate, it must be the same between what is in the code and what is at the bottom right of your interface.



The firmware waits for the command CAPTURE received through the serial interface. Upon reception, it captures an image and stores it locally.

---

5. LoRa TX Configuration

The steps are similar to the previous point. You must change the board, the COM (only if it’s necessary), the device (of course). So:

1. Open Arduino IDE.
2. Load LoRa_TX_Egg.ino (or the low_power version)
3. Upload.

The Communication parameters defined in the sketch are:

- Frequency: 868 MHz
- SF: 7
- BW: 125 kHz
- CR: 4/5
- Sync Word: 0x12

---

6. LoRa RX Configuration

1. Open LoRa_RX_Egg.ino (or the low_power version)
2. Upload.

Use exactly the same communication parameters configured on the transmitter. The steps of setup are the same.

---

7. AI Model

The repository already contains the model: modello_TRANSFER_ESP32CAM_egg.tflite

No additional training is required. It is already done on an appropriate dataset made by the Esp32Cam. If you want to create another model, you need to convert it in a tflite file at the end. If you want to create another model by using the same dataset, you can use the dataset_esp32_egg.zip and import it from the drive.

```
from google.colab import drive
drive.mount('/content/drive/')

dataset_zip_path = "/content/drive/MyDrive/dataset_esp32.zip"
dataset_path = "/content/dataset_esp32"

# Estrazione automatica dello zip, se necessario
if os.path.exists(dataset_zip_path) and not os.path.exists(dataset_path):
    with zipfile.ZipFile(dataset_zip_path, 'r') as zip_ref:
        zip_ref.extractall('/content')
```

To run the code, you can use Google Colab.

---

8. Configure Telegram

Create a Telegram Bot using @BotFather. Once you have done it, you will receive a TOKEN. Preserve it, we will use it on Node-RED.

---

9. Configure Node-RED

1. Start Node-RED on your terminal by writing node-red.cmd
2. Before importing the flow, install the required Node-RED packages through Manage Palette (Menu → Manage Palette → Install) or via terminal. Required packages:
   - node-red-dashboard
   - node-red-node-serialport
   - node-red-node-sqlite
   - node-red-contrib-telegrambot
3. Import flow_egg.json
4. Configure
   - Serial node, probably you have to change the serial port. Check it
   - SQLite node, you must change the path of the SQLite database with your own.
   - MQTT broker, check if you have already installed a broker to use locally. In the flow, the broker used is mosquitto. You can download it from https://mosquitto.org/download/
   - Telegram Sender, you must change the TOKEN with the one from your bot. You must change the chat ID on the previous function node too. To get it:
     - Put a telegram receiver node on the flow (it's not necessary it is linked with something)
     - Put a debug node after it and link the two nodes
     - Deploy and then write something on the chat with your bot
     - You can see the chat ID at the debug window on the right
5. Remove the Telegram receiver and the debug node, then Deploy.

---

10. Configure Cron

On your terminal, after having been connected to the raspberry by ssh, open crontab -e. It will require you the editor, choose nano. Insert the provided configuration contained inside crontab_egg.txt. The acquisition will automatically run at

- 08:00
- 11:00
- 15:00

Optionally, the Raspberry Pi can be configured to shut down at 15:15 by enabling the corresponding command in `crontab_egg.txt`. Make sure the Raspberry Pi system clock is correctly synchronized before running the scheduled tasks.So if you need to use it in a scenario without connection, connect the raspberry in advance for just few seconds with your hotspot.

---

11. Running the System

The complete pipeline is executed through python run_detection.py. The script automatically performs Capture -> Inference -> JSON generation -> LoRa transmission. Inside it there is the calling to two script: the first is the one which take the photo; the other it the one which allow the transmission.

---

12. Communication Protocol

The Generated payload is something similar to

{ "id":1718708423,

"egg":1,

"label":"Egg",

"prob":97.3 }

The Receiver ACK is something similar to

{"ack":1718708423}

The ACK contains the ID of the package in order to check the consistency.

---

13. Verify Correct Execution

The following components should operate correctly:

- ESP32-CAM captures images
- Raspberry receives images
- AI inference executes successfully
- LoRa packet transmitted
- ACK received
- Node-RED dashboard updated
- SQLite database updated
- Telegram notification generated

---

14. Troubleshooting

ESP32-CAM not responding

Verify

- serial port
- USB cable
- firmware upload

LoRa communication fails

Verify

- Frequency
- SF
- BW
- Coding Rate
- Sync Word

must match on both devices.

AI inference fails

Verify

modello_TRANSFER_ESP32CAM_egg.tflite

exists inside the repository.

Telegram notifications not received

Verify

- Bot Token
- Chat ID
- Internet connection

Node-RED dashboard not updating

Verify

- serial connection
- MQTT broker
- imported flow

---

15. Future Improvements

Possible improvements include:

- solar-powered Raspberry Pi;
- MPPT charger;
- Li-Ion battery;
- low-power scheduling;
- waterproof enclosure;
- remote firmware updates.

16. Little Tips

In order to manage all the hardware properly, you may use some protective cases. You can make them by yourself with a 3D-Print or buy it on internet.